SPADE: Speculative Decoding for Distributed Edge Cloud Inference

Preserving large-model accuracy while reducing cloud verification calls — a plug-and-play approach for edge cloud deployments

Divyajyoti Bajpai
Kishan Kumar Upadhyay
Manjesh Kumar Hanwal
IEOR, IIT Bombay

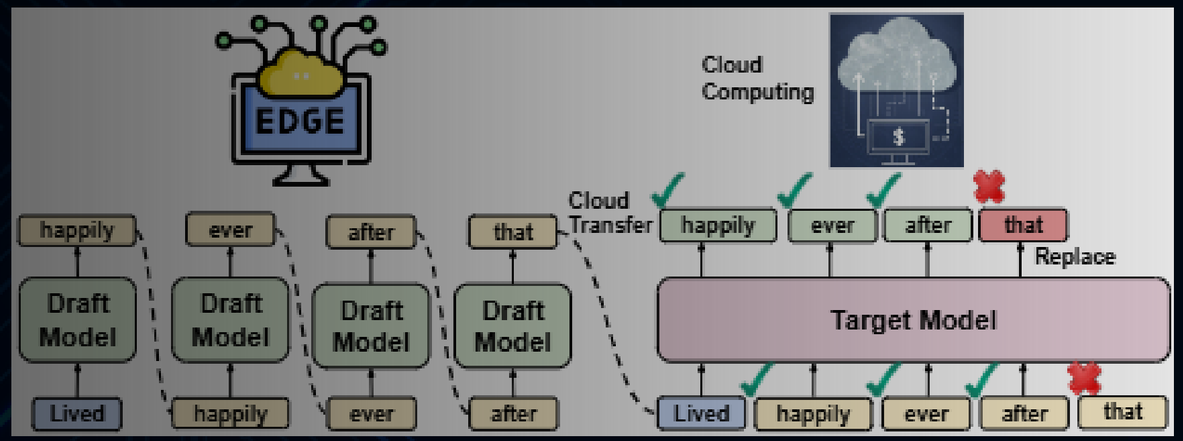
Contact:
divyajyoti.bajpai@iitb.ac.in
24n0453@iitb.ac.in
mhanawal@iitb.ac.in

THE CHALLENGE

- *Large Language Models (LLMs) require massive computational resources*
  - *Edge deployment: Small model, lower cost, but reduced accuracy*
  - *Cloud deployment: Large model, high accuracy, but expensive per-token computation*
- *Autoregressive decoding requires full forward pass for each token*
- *Core question: How to achieve high accuracy with low inference cost?*

Solution: Distributed inference combining edge and cloud resources

SPADE FRAMEWORK OVERVIEW



In this figure a lightweight edge model drafts tokens sequentially. A larger cloud verifier checks them in parallel; accepted tokens stay, rejects are replaced, and generation continues from the updated context.

Pipeline:

- Edge: Draft model generates block of d candidate tokens
- Cloud: Target model verify tokens in parallel
- Accept/reject tokens deterministically
- Resume generation from corrected context

Benefits:

- 76% reduction in cloud calls
- Zero accuracy loss
- No retraining required
- Plug-and-play design

ALGORITHM

Algorithm 1 Speculative Decoding for Distributed Inference

Input: Prompt $y_{1:m}$, Draft model $\mathcal{M}_q$ (edge), Verifier model $\mathcal{M}_p$ (cloud)
Initialize: $Y \leftarrow []$ ▷ Generated sequence
while $< eos >$ token not predicted **do**
  Edge: Generate d speculative tokens
  for $i \in [1, d]$ **do**
    $y_{m+i} \leftarrow \mathcal{M}_q(y_{1:m+i-1}) \sim q(\cdot)$
  end for
  Send $y_{m+1:m+d}$ **to cloud for verification**
  Cloud: In parallel, $[y^*_{m+i} \leftarrow \mathcal{M}_p(y_{1:m+i-1}) \sim p(\cdot)$ for $i \in [1, d+1]]$ in a single forward pass.
  Verify(y_{m+i}, y^*_{m+i}) in parallel.
  Accept the longest verified draft $y_{m:m+k}$
  Sample $\tilde{y}_{m+k+1} \leftarrow norm(max(0, p - q)$
  Correct the rejected draft $y_{m+k+1} = \tilde{y}_{m+k+1}$.
  Append tokens $y_{m:m+k+1}$ to Y
  Update context: $m \leftarrow m + k + 1$, $y_{1:m+k+1}$
end while
Return: Y

Key Parameter: Draft length d controls edge-cloud trade-off

EXPERIMENTAL SETUP

Hardware Configuration:

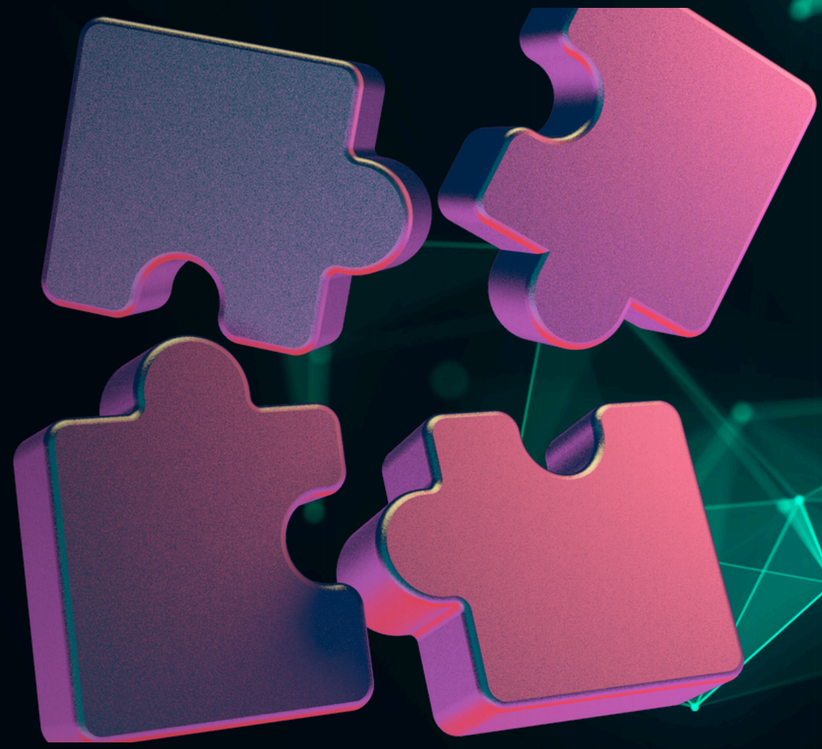
- Edge: NVIDIA RTX 3080 (12GB RAM)
- Cloud: NVIDIA RTX A6000 (48GB RAM)

Models:

- Draft: LLaMA-3.2-1B (edge deployment)
- Verifier: LLaMA-3.1-8B (cloud deployment)

Benchmarks:

- SpecBench: 6 NLP tasks (conversation, translation, summarization,RAG, Q&A, reasoning)
- CNN/DailyMail: Abstractive summarization



Evaluation Metrics:

- Performance: BLEU, ROUGE, CIDEr-D, LLM-as-judge scores
- Efficiency: Cloud model calls, tokens/sec, runtime reduction

RESULTS ON SPECBENCH

MEAN PERFORMANCE SCORES AND EFFICIENCY METRICS ACROSS TASKS ON THE SPEC-BENCH DATASET. THE RESULTS COMPARE THE Target, Draft, AND Speculative MODELS ACROSS MULTIPLE NLP TASKS. HIGHER VALUES (↑) INDICATE BETTER PERFORMANCE, WHILE LOWER VALUES (↓) INDICATE GREATER EFFICIENCY

Task / Metric	Target Model	Draft Model	Our Model (SPADE)
Task Scores (↑)			
Multi-turn Conversation	3.62	2.67	3.52
Translation	4.80	3.85	4.71
Summarization	4.61	4.41	4.55
Question Answering	4.25	3.08	4.15
Mathematical Reasoning	4.88	3.19	4.68
Retrieval-Augmented Generation	4.56	3.13	4.68
Overall Score (↑)	**4.45**	**3.39**	**4.38**
Efficiency Metrics			
Mean Target Model Calls (↓)	133.25	0.00	30.16
Average Throughput (tokens/s) (↑)	2.43	3.91	3.25
Cloud runtime (↓)	1.00×	–	0.23×

77.4% reduction in cloud model calls (30.16 vs 133.25)
Cloud runtime: 0.23× of full cloud mode

RESULTS ON CNN/ DAILYMAIL

PERFORMANCE COMPARISON ON THE CNN / DAILYMAIL SUMMARIZATION DATASET. HIGHER VALUES (↑) INDICATE BETTER GENERATION QUALITY.

Metric	Target Model	Draft Model	Our Model (SPADE)
BLEU-1 (↑)	23.76	22.33	23.39
BLEU-4 (↑)	07.57	06.49	06.98
ROUGE-1_{F1} (↑)	38.38	36.05	37.99
ROUGE-L_{F1} (↑)	24.32	22.49	23.92
CIDEr-D (↑)	02.50	01.15	03.19
Efficiency Metrics			
Mean Target Model Calls (↓)	127.30	0.00	30.79
Average Throughput (tokens/s) (↑)	1.21	2.82	1.95
Cloud runtime (↓)	1.00×	–	0.24×

76% reduction in cloud calls (30.79 vs 127.30)

Cloud runtime: 0.24× of full cloud model

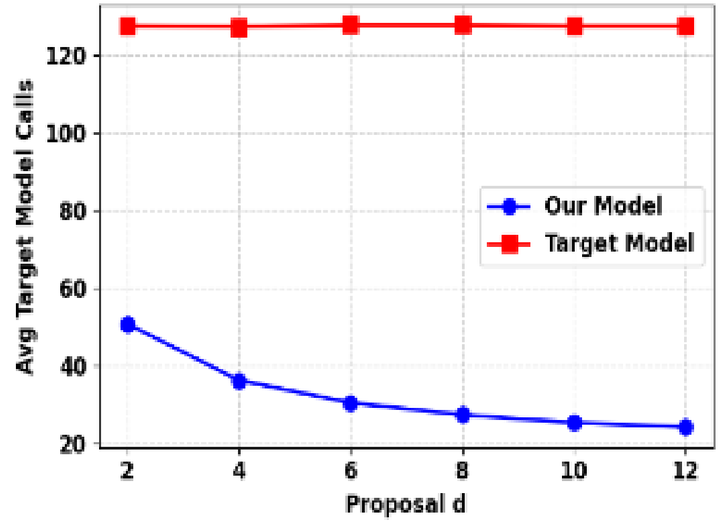


Fig. 2. Performance trade-off of our method, illustrating the variation in target model calls when the hyperparameter d (speculative token generation before one verifier call) increases.

SPADE VS EXISITING METHODS

Existing Distributed Inference Methods:

- Layer-splitting: Requires model retraining, breaks architecture
- Early-exit classifiers: Dataset-specific, poor cross-task generalization
- Complexity-aware routing: Overhead of complexity estimation

SPADE Unique Advantages:

1. First to combine speculative decoding + distributed inference
2. Theoretically guaranteed zero performance loss
3. Plug-and-play—no retraining required
4. Generalizes across tasks, datasets, and architectures
5. Simple design—minimal systems overhead

Core Innovation: Using speculative decoding as a resource allocation mechanism rather than just a speedup technique

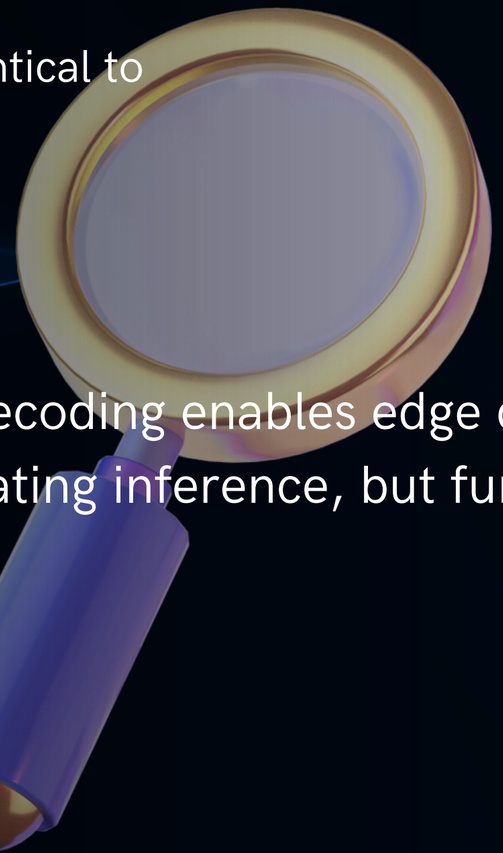
CONCLUSION

What We Accomplished:

- Distributed inference framework balancing accuracy and efficiency
- Reduced cloud computational burden by 76%
- Achieved faster cloud inference than full cloud model
- Zero accuracy loss—outputs identical to cloud-only deployment

Real-World Impact:

- Dramatically lower cloud API costs
- Lower latency through local edge processing
- Privacy benefits from on-device computation
- Scalable for large-scale LLM deployment



Link to our code: LINK

Speculative Decoding enables edge devices to intelligently assist cloud models—not just accelerating inference, but fundamentally changing deployment economics

THANK YOU!

FOR YOUR ATTENTION